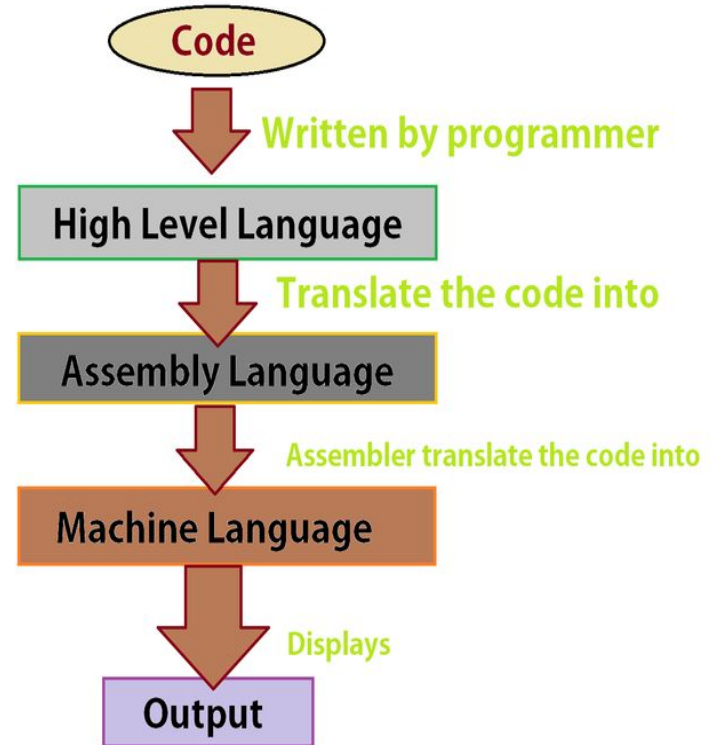


Introduction to Programming

What is Programming ?

2

- The act of writing program which a computer can understand and take appropriate action, is known as computer programming.
- Generally, it is a common set of sequenced instructions which defines specific tasks.
- There are a large number of high level languages, some of them are BASIC, C, C++, Java, Python, PHP, Pascal, Perl, Ruby, FORTRAN, and Visual Basic etc.
- These all comes under high level programming, which is then translated into assembly language to machine language, to be implemented on hardware and fetch output.



Python

3

- Python is an open-source (free) programming language that is used in web programming, data science, artificial intelligence, and many scientific applications.
- It is an interpreted, object oriented, high level programming language with dynamic semantics.
- It is very attractive for Rapid Application development.
- Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms, and can be freely distributed.

Why Python ?

4

Programs With Python

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Programs With Python IDLE

- Python Integrated Development and Learning Environment allows you to enter, edit, save, retrieve, compile, link, and run a python program.
- **Download the python IDE using the following link:**

<https://www.python.org/downloads/>

Get Starting with the jupyter notebook print command

 jupyter Untitled Last Checkpoint: 6 hours ago



File Edit View Run Kernel Settings Help

Trusted

 +        Code ▾

JupyterLab   Python 3 (ipykernel)  

```
[1]: print("Hello World")
```

```
Hello World
```

Escape Sequences

8

<u>Escape Sequence</u>	<u>Meaning</u>
\'	Single quote
\"	Double quote
\\	Backslash
\n	Newline
\r	Carriage Return/ raw string
\t	Horizontal Tab
\b	Backspace

Examples

9

- New line
- Backslash
- Tab space between word
- Escape single quotes
- To ignore all the escape sequences in the string, we have to make a string as a raw string using 'r' before the string

```
>>> print('daniel\nahmed')
daniel
ahmed
>>> print('daniel\\ahmed')
daniel\ahmed
>>> print('daniel\tahmed')
daniel  ahmed
>>> print('who\'s this')
who's this
>>> print(r'daniel\nahmed')
daniel\nahmed
```

Comments in Python

10

- # symbol is use to add the comments in python.
- It helps to understand the code to a new user or a new programmer who didn't know about it.
- Comments are very useful as it makes more understandable.



untitled

File Edit Format Run Options Window Help

```
print('This is Ali\'s book') # this sentence shows the possession  
print('Karachi, Pakistan') # this shows the location
```

Task

11

- Print the following output using escape characters:

```
# this is \\ double backslash
# these are /\ /\ /\ /\ mountains
# he is      awesome (use escape sequence instead of manual spaces )
# \" \n \t \' (print this as an output) I
```

Solution

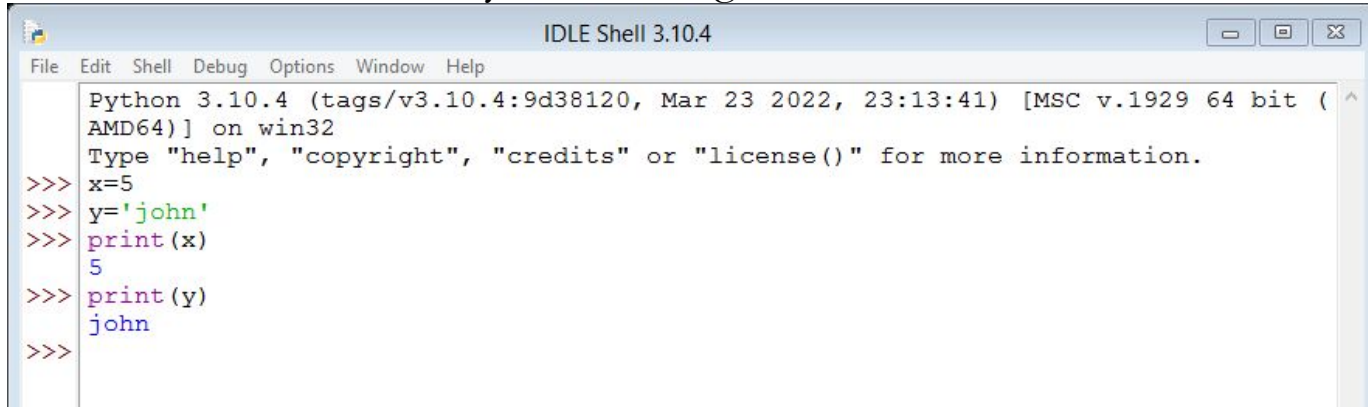
12

```
print("this is \\\\ double backslash")  
print("these are /\\/\\/\\/\\ mountains")  
print("he is\\tawesome")  
print(" \\\\" \\\n \\\t \\\\" ' ")
```

Python Variables and assignment

13

- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.

A screenshot of the IDLE Shell 3.10.4 window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the Python 3.10.4 prompt and the following code and output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> x=5
>>> y='john'
>>> print(x)
5
>>> print(y)
john
>>>
```

- Variables do not need to be declared with any particular type, and can even change type after they have been set.

```
x = 4          # x is of type int
x = "Sally"    # x is now of type str
print(x)
```

- There are different notions, symbols and parenthesis style to assign different data type:

Example	Data Type
<code>x = "Hello World"</code>	str
<code>x = 20</code>	int
<code>x = 20.5</code>	float
<code>x = 1j</code>	complex

String Operator

15

STRING

- Any data type surrounded by quotation mark (either single or double quotation) is called as string.

MULTILINE STRING

- Multiline string need multi quotation.

STRING LENGTH

- String length includes the alphabets and spaces as well.

CHECK STRING

- You can check string if it's present or not by using keyword 'in'

```
>>> a='string'
>>> print(a)
string
>>> a='''I
        love
        Pakistan'''
>>> print(a)
I
        love
        Pakistan
>>> a='I Love Pakistan'
>>> print(len(a))
15
>>> print('love' in a)
False
>>> print('Love' in a)
True
>>> |
```

Reading user input and Displaying Output

16

```
[4]: x = int(input("Enter first number"))  
     y = int(input("Enter Second number"))  
     z = x + y  
     print(z)
```

```
Enter first number 5  
Enter Second number 10  
15
```

int() function is use to convert a string or any other data type of integer data type

Reading user input and Displaying Output

17

- Take user input in one line using split method

```
name, age = input("Enter your name and age separated by comma : ").split(",")  
print(f"Name: {name}\n Age: {age}")
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
==== RESTART: C:/Users/User/Desktop/practise 1.py =====  
Enter your name and age separated by comma : Ali,10  
Name: Ali  
Age: 10
```

String Formatting

18

```
name = "Ali"  
age = 16  
print("hello " + name + " your age is " + str(age)) # Difficult to understand  
  
print("hello {} your age is {}".format(name, age)) # python 3
```

Output of the above two print command are same but the formatting is more clear in second one using the **format** method

```
>>> hello Aliyour age is 16  
hello Ali your age is 16  
>>> |
```

Ask user to input 3 numbers and you have to print the average of three numbers using string formatting.

```
1 num1 = input("enter first number :")
2 num2 = input("enter second number :")
3 num3 = input("enter third number :")
4 # (int(num1) + int(num2) + int(num3)) / 3
5 print(f"average of three numbers : {(int(num1) + int(num2) + int(num3)) / 3}")
```

Solution

20

```
1 num1 = input("enter first number :")
2 num2 = input("enter second number :")
3 num3 = input("enter third number :")
4 # (int(num1) + int(num2) + int(num3)) / 3
5 print(f"average of three numbers : {(int(num1) + int(num2) + int(num3)) / 3}")
```

```
enter first number :3
enter second number :3
enter third number :3
average of three numbers : 3.0
```

String Indexing

21

```
1  # string indexing
2  language = "python"
3
4  # positions(index number)
5
6  # p = 0
7  # y = 1
8  # t = 2
9  # h = 3
10 # o = 4
11 # n = 5
12
13 print(language[2])
14
```

t

```
1  # string indexing
2  language = "python"
3
4  # positions(index number)
5
6  # p = 0 , -6
7  # y = 1 , -5
8  # t = 2 , -4
9  # h = 3 , -3
10 # o = 4 , -2
11 # n = 5 , -1
12
13 print(language[-1])
14
```

n

String Slicing

22

Slicing / Selecting sub sequences

lang = "Python"

syntax -- [start argument : stop argument - 1]

print(lang[2:4])

```
>>>
=====
>>> th
>>>
```

Step Argument

23

```
# slicing / selecting sub sequences  
# lang = "Python"  
# syntax - [start argument : stop argument -1 : step]
```

```
lang = "Python"  
print(lang[0:5:2])
```

```
=====  
Pto  
>>>
```

- Reverse Slicing

```
lang = "Python"  
print(lang[::-1])
```

```
>>>  
nohtyP  
>>>
```

- Ask user name and print back user name in reverse order

Note: try to make your program in 2 lines using string formatting

Solution

25

```
name = input("enter your name : ")  
print(f"reverse of your name is {name[-1::-1]}")
```

String Methods

26

```
a = "Hello, World!"  
print(a.upper())
```

```
HELLO, WORLD!
```

```
a = "Hello, World!"  
print(a.lower())
```

```
hello, world!
```

```
a = " Hello, World! "  
print(a.strip())
```

```
Hello, World!
```

```
a = "Hello, World!"  
print(a.replace("H", "J"))
```

```
Jello, World!
```

```
a = "Hello, World!"  
print(a.split(",")) # returns ['Hello', ' World!']
```

```
['Hello', ' World!']
```

```
a = "Hello, World!"  
print(len(a))
```

12

String Methods

27

```
a = "Hello, World!"  
print(a.count('H'))
```

1

- Take two inputs as comma separated values from user and a single character from the name and count the character presence in the given name.

```
name, char = input("enter comma separated name and character : ").split(",")  
print(f"length of your name is {len(name)}")  
print(f"character count : {name.count(char)}") # case sensitive
```

```
enter comma separated name and character : Harshit,h  
length of your name is 7  
character count : 1
```

Solution

29

```
name, char = input("enter comma separated name and character : ").split(",")  
print(f"length of your name is {len(name)}")  
print(f"character count : {name.count(char)}") # case sensitive
```

```
enter comma separated name and character : Harshit,h  
length of your name is 7  
character count : 1
```

Python Operators

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators

Arithmetic Operators

32

- It deals with numeric values and common mathematical operations

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Comparison Operators

33

- Comparison between two values can be made by using comparison operators.

Operator	Name	Example
==	Equal	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>

Logical Operators

34

- Conditional statements requires logical operators.

Operator	Description	Example
and	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	not($x < 5$ and $x < 10$)

Identity Operators

35

- Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Membership Operators

36

- Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Basic arithmetic operator examples

37

Add

Subtract

Multiply

Divide

Floor Division

Exponent

```
x = 15
```

```
y = 4
```

```
# Output: x + y = 19
```

```
print('x + y =',x+y)
```

```
# Output: x - y = 11
```

```
print('x - y =',x-y)
```

```
# Output: x * y = 60
```

```
print('x * y =',x*y)
```

```
# Output: x / y = 3.75
```

```
print('x / y =',x/y)
```

```
# Output: x // y = 3
```

```
print('x // y =',x//y)
```

```
# Output: x ** y = 50625
```

```
print('x ** y =',x**y)
```

Output

```
x + y = 19
```

```
x - y = 11
```

```
x * y = 60
```

```
x / y = 3.75
```

```
x // y = 3
```

```
x ** y = 50625
```

Comparison operator examples

38

Greater than

Less than

Equal to

Not Equal to

Greater than or equal to

Less than or equal to

```
x = 10
y = 12

# Output: x > y is False
print('x > y is',x>y)

# Output: x < y is True
print('x < y is',x<y)

# Output: x == y is False
print('x == y is',x==y)

# Output: x != y is True
print('x != y is',x!=y)

# Output: x >= y is False
print('x >= y is',x>=y)

# Output: x <= y is True
print('x <= y is',x<=y)
```

Output

```
x > y is False
x < y is True
x == y is False
x != y is True
x >= y is False
x <= y is True
```

Logical operator examples

39

And

Or

Not

```
x = True
y = False

print('x and y is', x and y)

print('x or y is', x or y)

print('not x is', not x)
```

Output

```
x and y is False
x or y is True
not x is False
```

Identity operator examples

40

Is

Is not

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
x3 = [1,2,3]
y3 = [1,2,3]

# Output: False
print(x1 is not y1)

# Output: True
print(x2 is y2)

# Output: False
print(x3 is y3)
```

Output

```
False
True
False
```

Membership operator examples

41

In

```
x = 'Hello world'  
y = {1:'a',2:'b'}
```

```
# Output: True  
print('H' in x)
```

In not

```
# Output: True  
print('hello' not in x)
```

```
# Output: True  
print(1 in y)
```

```
# Output: False  
print('a' in y)
```

Output

```
True  
True  
True  
False
```

square root.py - C:/Users/Hera Noor/AppData/Local/Programs/Python/Python38/square root

Edit Format Run Options Window Help

```
= float(input('Enter a number: '))  
_sqrt = num ** 0.5  
nt('The square root of %0.3f is %0.3f'%(num ,num_sqrt))
```

 IDLE Shell 3.8.10

File Edit Shell Debug Options Window Help

Python 3.8.10 (tags/v3.8.10:3d8993a, May
AMD64)] on win32

Type "help", "copyright", "credits" or "li

>>>

= RESTART: C:/Users/Hera Noor/AppData/Loca

.py

Case Sensitive

- Python is case sensitive; you can declare a variable with 'x' and a different variable with 'X'.

Quotes

- Single and double quotes are same.
- It is usually used to declare string variables.

Casting

44

- The method to declare any variable as any particular type, is called casting.
- The type can also be changed after it has been set.

```
x = str(3)    # x will be '3'  
y = int(3)    # y will be 3  
z = float(3)  # z will be 3.0
```

Get The Type

- The type can be seen by using

```
>>> x=5  
>>> y='john'  
>>> print(x)  
5  
>>> print(y)  
john  
>>> print(type(x))  
<class 'int'>  
>>> print(type(y))  
<class 'str'>  
>>>
```

-
- Take three input from user i.e. name, phone number and Address, display it on the output with appropriate sentence using escape character and right data type
 - Take marks and subject name as an input from the user and display it in an appropriate format.

Three different number type is present in python.

- Int – Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.
- Float – Float, or "floating point number" is a number, positive or negative, containing one or more decimals. Float can also be scientific numbers with an "e" to indicate the power of 10.
- Complex – Complex numbers are written with a "j" as the imaginary part.

Conversion of Types

47

- You can convert from one type to another with the `int()`, `float()`, and `complex()` methods.
- Considering the same example:

Conversion

Output

```
x=2
y=7.9
z=complex(4,5)
print(x)
print(y)
print(z)
a=float(x)
b=int(y)
c=complex(x)
print(type(a))
print(type(b))
print(type(c))
|
```

```
2
7.9
(4+5j)
<class 'float'>
<class 'int'>
<class 'complex'>
>>> |
```

- Indentation refers to the spaces at the beginning of a code line.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses indentation to indicate a block of code.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python will show error if you miss the indent.

```
if 5 > 2:  
    print("Five is greater than two!")
```

Syntax Error:

```
if 5 > 2:  
print("Five is greater than two!")
```

- The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

```
if 5 > 2:  
    print("Five is greater than two!")  
if 5 > 2:  
    print("Five is greater than two!")
```

- You have to use the same number of spaces in the same block of code, otherwise Python will give you an error:

Syntax Error:

```
if 5 > 2:  
    print("Five is greater than two!")  
    print("Five is greater than two!")
```

Decision Making and Conditional Operator

Decision Making Structures

Normally, the program flows along line by line in the order in which it appears in source code. But, it is sometimes required to execute a particular portion of code only if certain condition is true; or false i.e. you have to make decision in the program.

General Format

```
if test1:                # if test
    statements1           # Associated block
elif test2:              # Optional elifs
    statements2
else:                     # Optional else
    statements3
```

Python Conditions and If statements

52

Python supports the usual logical conditions from mathematics:

Equals: $a == b$

Not Equals: $a != b$

Less than: $a < b$

Less than or equal to: $a \leq b$

Greater than: $a > b$

Greater than or equal to: $a \geq b$

These conditions can be used in several ways, most commonly in "if statements" and loops.

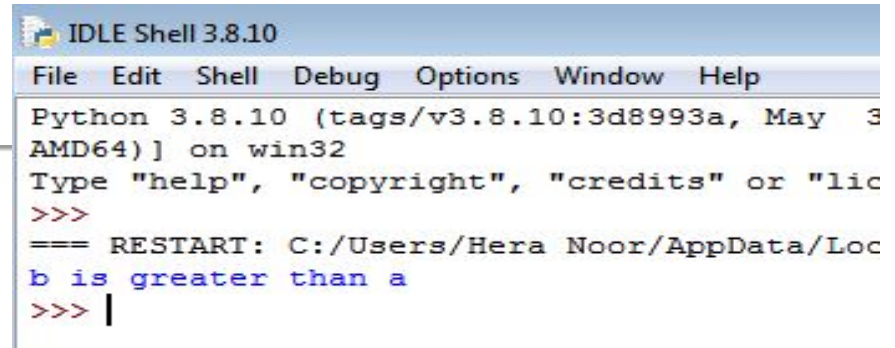
An "if statement" is written by using the if keyword.

Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

CODE

OUTPUT

```
a = 86
b = 300
if b > a:
    print("b is greater than a")
```



```
IDLE Shell 3.8.10
File Edit Shell Debug Options Window Help
Python 3.8.10 (tags/v3.8.10:3d8993a, May 3
AMD64)] on win32
Type "help", "copyright", "credits" or "lic
>>>
=== RESTART: C:/Users/Hera Noor/AppData/Loc
b is greater than a
>>> |
```

The **'elif'** keyword is python's way of saying "if the previous conditions were not true, then try this condition".

CODE

OUTPUT

```
File Edit Format Run Options Window Help
```

```
a = int(input('enter first number'))  
b = int(input('enter second number'))  
if b>a:  
    print("b is greater than a")  
elif a>b:  
    print("a is greater than b")  
|
```

```
=== RESTART: C:/Users/Hera 1  
enter first number32  
enter second number12  
a is greater than b  
>>> |
```

It is used as the final condition respective to 'if' operator. Note: no condition can be defined within else statement.

CODE

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

OUTPUT

```
==== RESTART: C:/Users/Hera Noor/A
b is not greater than a
>>> |
```

And & Or

56

The and & Or keyword is a logical operator, and is used to combine conditional statements

CODE

OUTPUT

 practise.py - C:/Users/User/Desktop/practise.py (3.13.7)

File Edit Format Run Options Window Help

```
a = int(input('enter first number'))
b = int(input('enter second number'))
c = int(input('enter third number'))

if a > b and b > c:
    print('Both conditions are True')
elif a > b or b > c:
    print('Atleast one of the condition is True')
else:
    print('the condition is false')
```

===== RESTART

enter first number100

enter second number80

enter third number60

Both conditions are True

enter first number100

enter second number120

enter third number80

Atleast one of the condition is True

Pass statement

'if' statements cannot be empty, but 'if' you for some reason have an 'if' statement with no content, put in the 'pass statement' to avoid getting an error.

```
a = 33  
b = 200  
  
if b > a:  
    pass
```

Task: Design a simple calculator using conditional statement

Iterative Structure (for and while)

'for' loop

59

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc. The for loop does not require an indexing variable to set beforehand.

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

Break & Continue Statement

60

With the break statement we can stop the loop before it has looped through all the items: Exit the loop when x is "banana":

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

```
=== RESTART: C:/Users/Hei
apple
banana
>>>
```

With the continue statement we can stop the current iteration of the loop, and continue with the next:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

```
=== RESTART: C:/U
apple
cherry
>>>
```

The range() Function

61

To loop through a set of code a specified number of times, we can use the range() function. The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Note that range(6) is not the values of 0 to 6, but the values 0 to 5.

```
>>> for x in range(6):  
    print(x)
```

```
0  
1  
2  
3  
4  
5
```

```
>>> for x in range(2, 30, 3):  
    print(x)
```

```
2  
5  
8  
11  
14  
17  
20  
23  
26  
29  
>>> |
```

Nested Loops

62

A nested loop is a loop inside a loop.

The "inner loop" will be executed one time for each iteration of the "outer loop":

Example

Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
    for y in fruits:
        print(x, y)
```

```
=== RESTART: C:/Users/He
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
>>>
```

The pass Statement

63

‘For’ loops cannot be empty, but if you for some reason have a ‘for’ loop with no content, put in the ‘pass’ statement to avoid getting an error.

Example

```
for x in [0, 1, 2]:  
    pass
```

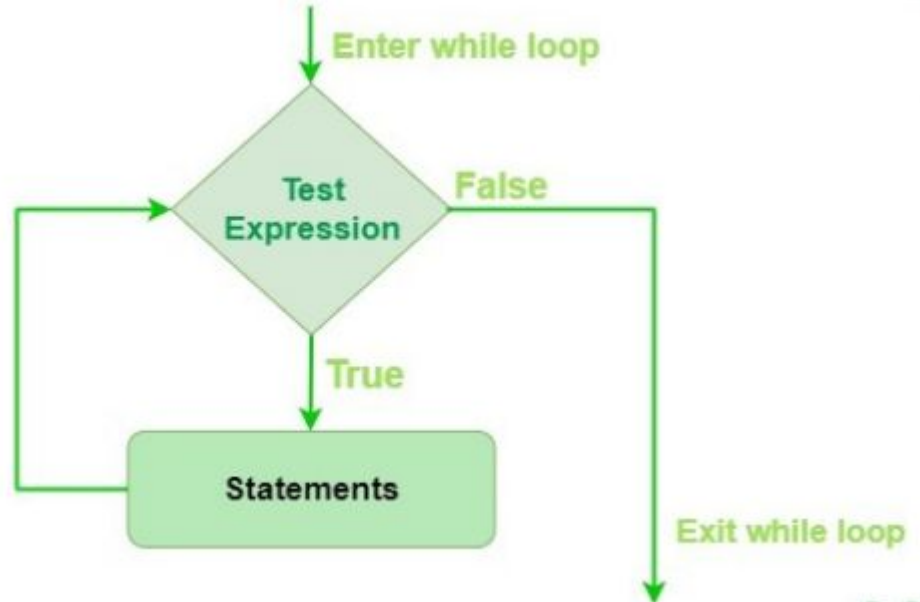
'while' loop

64

Python While Loop is used to execute a block of statements repeatedly until a given condition is satisfied. And when the condition becomes false, the line immediately after the loop in the program is executed.

Syntax:

```
while expression:  
    statement(s)
```



Example

Print i as long as i is less than 6:

Note: remember to increment i, or else the loop will continue forever. The while loop requires relevant variables to be ready, in this example we need to define an indexing variable, i, which we set to 1.

```
i = 1
while i < 6:
    print(i)
    i += 1
```

```
=== RESTART: C:/Users/Hera
1
2
3
4
5
>>>
```

The break & Continue Statement

66

With the break statement we can stop the loop even if the while condition is true:

Example

Exit the loop when i is 3:

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

```
== RESTART:
1
2
3
>>> |
```

With the continue statement we can stop the current iteration, and continue with the next:

Example

Continue to the next iteration if i is 3:

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

```
== RESTART: C:
1
2
4
5
6
>>>
```

The else Statement

67

With the else statement we can run a block of code once when the condition no longer is true:

Example

Print a message once the condition is false:

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

```
== RESTART: C:/Users/Hera Noor,
1
2
3
4
5
i is no longer less than 6
>>> |
```

Task

68

- Challenge 1: Write a program that takes a number as input and checks whether it is even or odd.
- Challenge 2: Write a program that prints the multiplication table of a number entered by the user.
- Challenge 3: Print a half and full pyramid using loop

Solution to the challenges

Challenge 1:

```
python

number = int(input("Enter a number: "))

if number % 2 == 0:
    print(number, "is even.")
else:
    print(number, "is odd.")
```

Challenge 2:

```
python

number = int(input("Enter a number: "))

print("Multiplication Table of", number)
for i in range(1, 11):
    print(number, "x", i, "=", number * i)
```

Printing pyramids using loops

▶ Half Pyramid:

```
rows = int(input("Enter the number of rows: "))

for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

• Full pyramid:

```
rows = int(input("Enter the number of rows: "))

for i in range(1, rows + 1):
    print(" " * (rows - i), end="")
    print("* " * i)
```

Data Type – List

Data types – List

72

List is a collection which is ordered and changeable. Allows duplicate members.

Multiple items can be stored on a single variable. For creation of list, we use square brackets [], and every item must be in single ' ' or double " " quotation mark.

```
>>> x=['a','b','c','d']
>>> print(x)
['a', 'b', 'c', 'd']
>>> |
```

Python lists have following basic properties:

- Ordered collections of arbitrary objects

From a functional view, lists are just places to collect other objects. Lists also maintain a **left-to-right positional ordering** among the items contained in them (i.e., they are sequences).

- Accessed by offset

A component object of list can be accessed by its position.

- Of the category “mutable sequence”

Lists are **mutable** (i.e., can be changed in place)

Indexing

74

The indexing is processed in the same way as it happens with the strings. The elements of the list can be accessed by using the slice operator `[]`. The index starts from 0 and goes to length - 1. The first element of the list is stored at the 0th index, the second element of the list is stored at the 1st index, and so on.

List = [0, 1, 2, 3, 4, 5]

0	1	2	3	4	5
---	---	---	---	---	---

List[0] = 0

List[0:] = [0,1,2,3,4,5]

List[1] = 1

List[:] = [0,1,2,3,4,5]

List[2] = 2

List[2:4] = [2, 3]

List[3] = 3

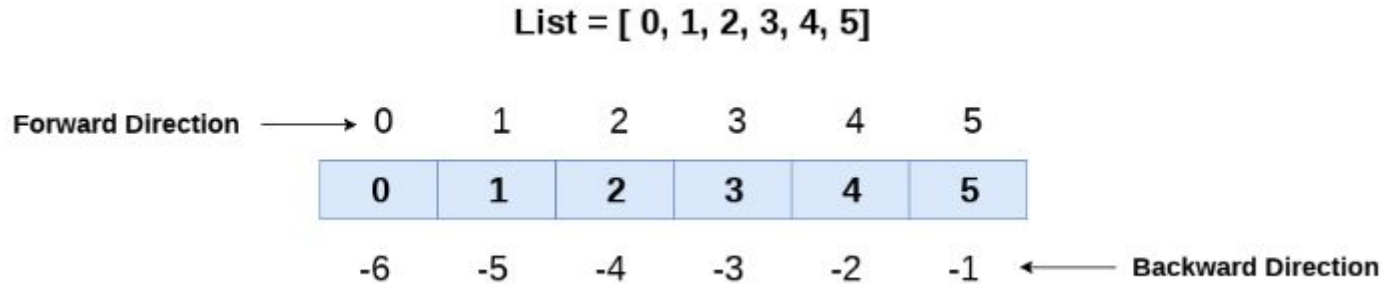
List[1:3] = [1, 2]

List[4] = 4

List[:4] = [0, 1, 2, 3]

List[5] = 5

Python provides the flexibility to use the negative indexing also. The negative indices are counted from the right. The last element (rightmost) of the list has the index -1; its adjacent left element is present at the index -2 and so on until the left-most elements are encountered.



Access Item using Indexing

76

'L' is the variable which contain a group of items.

List can contain different type of data, as you can see.

L[2] – is an offset which starts from zero, and designated offset item is called.

Negative offset counts from the right i.e. reverse, but not from zero.

Len() – determines the length of list.

From Python's perspective, lists are defined as objects with the data type 'list'

```
L = ['apple', 'mango', '1', '2', '3']
print(L)
print(L[4])
print(L[-3])
print(len(L))
print(type(L))
```

```
===== RESTART: C:/Us
['apple', 'mango', '1', '2', '3']
3
1
5
<class 'list'>
```

List Operations

77

The concatenation (+) and repetition (*) operators work in the same way as they were working with the strings. **Consider a Lists l1 = [1, 2, 3, 4], and l2 = [5, 6, 7, 8] to perform operation.**

Operator	Description	Example
Repetition	The repetition operator enables the list elements to be repeated multiple times.	<code>l1*2 = [1, 2, 3, 4, 1, 2, 3, 4]</code>
Concatenation	It concatenates the list mentioned on either side of the operator.	<code>l1+l2 = [1, 2, 3, 4, 5, 6, 7, 8]</code>
Membership	It returns true if a particular item exists in a particular list otherwise false.	<code>print(2 in l1)</code> prints True.
Iteration	The for loop is used to iterate over the list elements.	<pre>for i in l1: print(i) Output 1 2 3 4</pre>

Check Item presence in List

78

To determine if a specified item is present in a list use the 'in' keyword.

```
File Edit Format Run Options Window Help
L=['apple', 'banana', 'mango', 'apple', '1', '2', '3', '4', 'a', 'b', 'c', 'd']
if "peach" in L:
    print("Yes, it is in the list")
else:
    print ('not in the list')
|
```

```
==== RESTART: C:/Users/Hera No
Yes, 'apple' is in the list
>>>
==== RESTART: C:/Users/Hera No
not in the list
>>> |
```

Change Item value

79

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values. If you insert more items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly.

```
>>> L
['apple', 'blackberry', 'mango', 'apple', '1', '2', '3', '4', 'a', 'b', 'c', 'd']
>>> L[1]="peach"
>>> L
['apple', 'peach', 'mango', 'apple', '1', '2', '3', '4', 'a', 'b', 'c', 'd']
>>> L[1:3]=["banana","z"]
>>> L
['apple', 'banana', 'z', 'apple', '1', '2', '3', '4', 'a', 'b', 'c', 'd']
>>> L[1:3]=["watermelon"]
>>> L
['apple', 'watermelon', 'apple', '1', '2', '3', '4', 'a', 'b', 'c', 'd']
>>> |
```

List Functions

80

The **insert()** method inserts an item at the specified index.

```
L = ['apple', 'mango', '1', '2', '3']  
print(L)  
L.insert(2, 'apricot')  
print(L)
```

>>>

```
===== RESTART: C:/Users/user/De  
['apple', 'mango', '1', '2', '3']  
['apple', 'mango', 'apricot', '1', '2', '3']
```

To add an item to the end of the list, use the **append()** method.

```
L = ['apple', 'mango', '1', '2', '3']  
print(L)  
L.append('orange')  
print(L)
```

>>>

```
===== RESTART: C:/Users/user/De  
['apple', 'mango', '1', '2', '3']  
['apple', 'mango', '1', '2', '3', 'orange']
```

Extend() method used to extend the item at the end of the list. It take input in the form of list.

```
L = ['apple', 'mango', '1', '2', '3']  
print(L)  
L.extend(['a', 'b'])  
print(L)
```

>>>

```
['apple', 'mango', '1', '2', '3']  
['apple', 'mango', '1', '2', '3', 'a', 'b']
```

List Functions

81

The **remove()** method remove an item from the list.

```
L = ['apple', 'mango', '1', '2', '3']  
print(L)  
L.remove('mango')  
print(L)
```

>>>

```
===== RESTART: C:/Users/  
['apple', 'mango', '1', '2', '3']  
['apple', '1', '2', '3']
```

The **pop()** method removes the specified index. If you do not specify the index, the **pop()** method removes the last item.

```
L = ['apple', 'mango', '1', '2', '3']  
print(L)  
L.pop(2)  
print(L)
```

>>>

```
===== RESTART: C:/Users/  
['apple', 'mango', '1', '2', '3']  
['apple', 'mango', '2', '3']
```

The **del** keyword also removes the specified index. The del keyword can also delete the list

```
L = ['apple', 'mango', '1', '2', '3']  
print(L)  
del L  
print(L)
```

```
['apple', 'mango', '1', '2', '3']
```

```
Traceback (most recent call last):
```

```
File "C:/Users/User/Desktop/practise.py", line 4, in <module>  
    print(L)
```

```
NameError: name 'L' is not defined
```

The **clear()** method empties the list. The list still remains, but it has no content. **L.clear()**

Copy and Join List

82

```
>>> L
['a', 'apple', 'c', 'd', 'e', 'f', 'g', 'orange']
>>> A=L.copy()
>>> A
['a', 'apple', 'c', 'd', 'e', 'f', 'g', 'orange']
>>> B=list(A)
>>> B
['a', 'apple', 'c', 'd', 'e', 'f', 'g', 'orange']
>>> |
```

```
>>> B
['a', 'apple', 'c', 'd', 'e', 'f', 'g', 'orange']
>>> C=['lion', 'tiger', 'cat', 'dog']
>>> D=B+C
>>> D
['a', 'apple', 'c', 'd', 'e', 'f', 'g', 'orange', 'lion', 'tiger', 'cat', 'dog']
>>> |
```

Python provides the following built-in functions, which can be used with the lists.

Function	Description	Example
<code>cmp(list1, list2)</code>	It compares the elements of both the lists.	This method is not used in the Python 3 and the above versions.
<code>len(list)</code>	It is used to calculate the length of the list.	<code>L1 = [1,2,3,4,5,6,7,8]</code> <code>print(len(L1)) 8</code>
<code>max(list)</code>	It returns the maximum element of the list.	<code>L1 = [12,34,26,48,72]</code> <code>print(max(L1)) 72</code>

<code>min(list)</code>	It returns the minimum element of the list.	<code>L1 = [12,34,26,48,72]</code> <code>print(min(L1))</code> 12
<code>list(seq)</code>	It converts any sequence to the list.	<code>str = "Johnson"</code> <code>s = list(str)</code> <code>print(type(s))</code> <class list>

List method call - Summary

85

Some built-in python methods, we have gone through are summarized here.

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list

<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list

Data types - Tuple

A tuple is a collection which is ordered and unchangeable.

Tuples are written with round brackets.

Tuple items are ordered, unchangeable, and allow duplicate values.

Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Creating a Tuple

89

A tuple can be written as the collection of comma-separated (,) values enclosed with the small () brackets. The parentheses are optional but it is good practice to use.

```
>>> T1 = (101, "Peter", 22)
>>> T2 = ("Apple", "Banana", "Orange")
>>> T3 = 10,20,30,40,50
>>> print(type(T1))
<class 'tuple'>
>>> print(type(T2))
<class 'tuple'>
>>> print(type(T3))
<class 'tuple'>
```

Update Tuple

90

Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created. Once a tuple is created, you cannot change its values. Tuples are unchangeable, or immutable as it also is called.

But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

```
>>> L=list(T1)
>>> L
['apple', 'banana', 'cherry', 'orange', 'kiwi', 'melon', 'mango']
>>> print (type(L))
<class 'list'>
>>> x=tuple(L)
>>> x
('apple', 'banana', 'cherry', 'orange', 'kiwi', 'melon', 'mango')
>>> print(type(x))
<class 'tuple'>
>>> |
```

Data Type – Sets

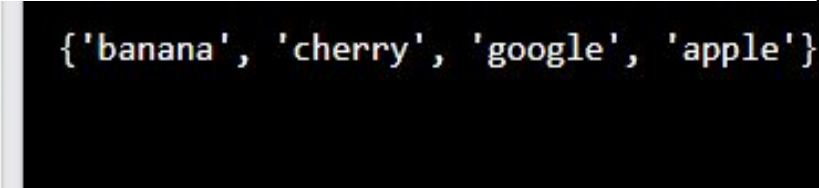
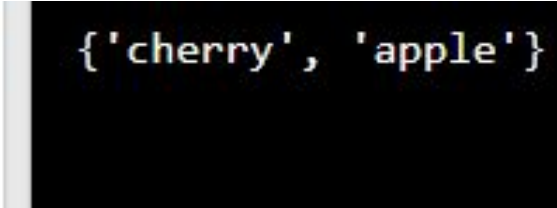
- ❖ Sets are used to store multiple items in a single variable.
- ❖ A set is a collection which is unordered, Changeable, unindexed and does not allow duplicate values.
- ❖ Unordered means that the items in a set do not have a defined order.
- ❖ Changeable, meaning that we can change the items after the set has been created but we can add new items or remove the existing ones.
- ❖ Sets cannot have two items with the same value.
- ❖ Most of the methods like `len()`, `type()` are same as discussed in the other data types.
- ❖ The `Set()` constructor is used to convert it into the set data type.

Method	Description
<u>add()</u>	Adds an element to the set
<u>update()</u>	Update the set with the union of this set and others
<u>difference()</u>	Returns a set containing the difference between two or more sets
<u>remove()</u>	Removes the specified element, Raise error if element does not exist in set
<u>discard()</u>	Remove the specified item, No error occur if item doesn't exist
<u>pop()</u>	Removes the first element from the set. <i>(No indexing allowed in it)</i>
<u>union()</u>	Return a set containing the union of sets
<u>intersection()</u>	Returns a set, that is the intersection of two other sets

Method	Example
<u>add()</u>	<pre>thisset = {"apple", "banana", "cherry"} thisset.add("orange") print(thisset)</pre> { 'orange', 'apple', 'banana', 'cherry' }
<u>update()</u>	<pre>x = {"apple", "banana", "cherry"} y = {"google", "microsoft", "apple"} x.update(y) print(x)</pre> { 'banana', 'apple', 'cherry', 'google', 'microsoft' }

Method	Example
<u>difference()</u>	<pre data-bbox="388 339 1093 547">x = {"apple", "banana", "cherry"} y = {"google", "microsoft", "apple"} z = x.difference(y) print(z)</pre> <pre>{'banana', 'cherry'}</pre>
<u>remove()</u>	<pre data-bbox="372 620 1078 929">fruits = {"apple", "banana", "cherry"} fruits.remove("banana") print(fruits) fruits = {"apple", "banana", "cherry"} fruits.remove("orange") print(fruits)</pre> <pre>{'apple', 'cherry'}</pre> <pre>Traceback (most recent call last): File "./prog.py", line 2, in <module> KeyError: 'orange'</pre>

Method	Description
<u>discard()</u>	<pre>fruits = {"apple", "banana", "cherry"} fruits.discard("banana") print(fruits)</pre> <pre>fruits = {"apple", "banana", "cherry"} fruits.discard("orange") print(fruits)</pre> <pre>{'cherry', 'apple'}</pre><pre>{'banana', 'cherry', 'apple'}</pre>
<u>pop()</u>	<pre>fruits = {"apple", "banana", "cherry"} fruits.pop() print(fruits)</pre> <pre>{'banana', 'cherry'}</pre>

Method	Description
<u>union()</u>	<pre>x = {"apple", "banana", "cherry"} y = {"google", "cherry", "apple"} z = x.union(y) print(z)</pre>  A black rectangular box with a light gray vertical bar on the left side, containing the text <code>{'banana', 'cherry', 'google', 'apple'}</code> in a white monospace font.
<u>intersection()</u>	<pre>x = {"apple", "banana", "cherry"} y = {"google", "cherry", "apple"} z = x.intersection(y) print(z)</pre>  A black rectangular box with a light gray vertical bar on the left side, containing the text <code>{'cherry', 'apple'}</code> in a white monospace font.

Data types - Dictionaries

Dictionary in Python is a collection of keys values, used to store data values like a map, which, unlike other data types which hold only a single value as an element. The dictionary is defined into element Keys and values.

- Keys must be a single element
- Value can be any type such as list, tuple, integer, etc.

A dictionary is an associative array (also known as hashes). Any key of the dictionary is associated (or mapped) to a value. The values of a dictionary can be any Python data type. So dictionaries are unordered key-value-pairs.

Create a Dictionary

100

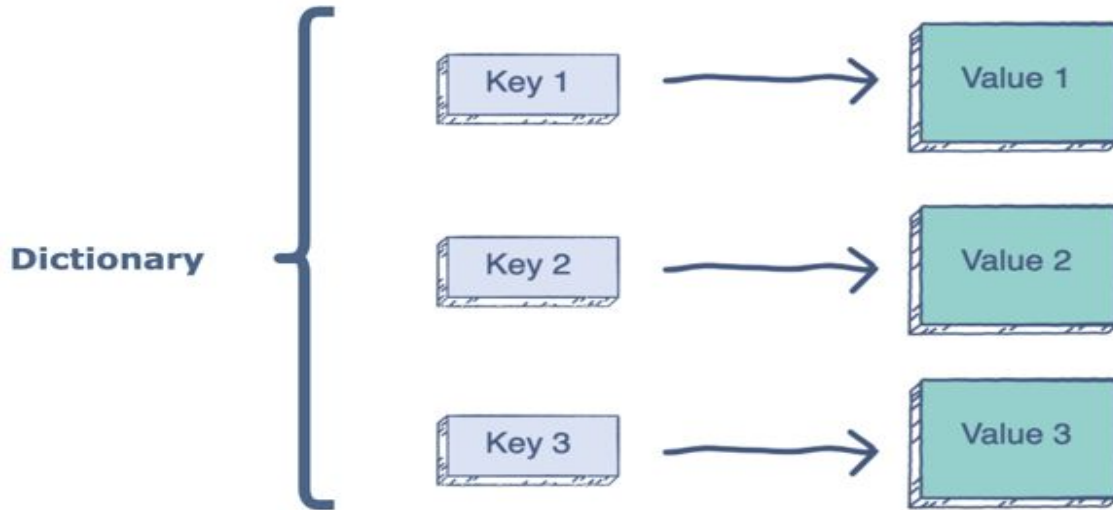
In Python, a dictionary can be created by placing a sequence of elements within curly {} braces, separated by 'comma'. Dictionary holds pairs of values, one being the Key and the other corresponding pair element being its Key value. Dictionary can also be created by the built-in function dict().

Note – Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

```
>>> dict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
>>> dict  
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}  
>>> |
```

How does the dictionary works in Python?

Each key is associated with a single value. The association of a key and a value is called a key-value pair or an item. Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type (i.e., strings, numbers, or tuples).



The values in dictionary items can be of any data type. Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

```
>>> dict = {  
    "1": "apple",  
    "2": False,  
    "3": 1964  
}  
>>> dict  
{'1': 'apple', '2': False, '3': 1964}  
>>> print(dict['1'])  
apple  
>>> |
```

When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change. Unordered means that the items does not have a defined order, you cannot refer to an item by using an index. **Dictionaries cannot have two items with the same key.**

Dictionary Length

103

To determine how many items a dictionary has, use the **len()** function:

```
>>> dict
{'1': 'apple', '2': 'mani', '3': 1964}
>>> print(dict['1'])
apple
>>> print(len(dict))
3
>>> |
```

type()

From Python's perspective, dictionaries are defined as objects with the data type 'dict'.

```
>>> print(type(dict))
<class 'dict'>
>>> |
```

1. The items of a dictionary can be accessed by referring to its key name, inside square brackets.
2. Another method called to access is **get()** that will give you the same result.

```
>>> dict
{'1': 'corolla', '2': 'alto', '3': 'ferrari', '4': 'altis', '5': 'cultus', '6':
'city', '7': '1993'}
>>> x=dict['5']
>>> x
'cultus'
>>> y=dict.get('4')
>>> y
'altis'
>>> |
```

Get Keys

105

The **keys()** method will return a list of all the keys in the dictionary. The list of the keys is a view of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.

Add a new item to the original dictionary, and see that the keys list gets updated as well.

```
>>> x=dict.keys()
>>> x
dict_keys(['1', '2', '3', '4', '5', '6', '7'])
>>> dict['8']='Alto VXL'
>>> dict
{'1': 'corolla', '2': 'alto', '3': 'ferrari', '4': 'altis', '5': 'cultus', '6': 'city', '7': '1993', '8': 'Alto VXL'}
>>> x
dict_keys(['1', '2', '3', '4', '5', '6', '7', '8'])
>>> [
```

The **values()** method will return a list of all the values in the dictionary. The list of the values is a view of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list. Make a change in the original dictionary, and see that the values list gets updated, or add a new item to the original dictionary, and see that the values list gets updated as well.

```
>>> a=dict.values()
>>> a
dict_values(['corolla', 'alto', 'ferrari', 'altis', 'cultus', 'city', '1993', 'A
lto VXL'])
>>> dict['year']=2022
>>> a
dict_values(['corolla', 'alto', 'ferrari', 'altis', 'cultus', 'city', '1993', 'A
lto VXL', 2022])
>>> |
```

The **items()** method will return each item in a dictionary, as tuples in a list.

The returned list is a view of the items of the dictionary, meaning that any changes done to the dictionary will be reflected in the items list. Make a change or add a new item in the original dictionary, and see that the items list gets updated as well.

```
>>> b=dict.items()
>>> b
dict_items([('1', 'corolla'), ('2', 'alto'), ('3', 'farrari'), ('4', 'altis'), ('5', 'cultus'), ('6', 'city'), ('7', '1993'), ('8', 'Alto VXL'), ('year', 2022)])
>>> dict['year']=1947
>>> b
dict_items([('1', 'corolla'), ('2', 'alto'), ('3', 'farrari'), ('4', 'altis'), ('5', 'cultus'), ('6', 'city'), ('7', '1993'), ('8', 'Alto VXL'), ('year', 1947)])
>>> |
```

Check existence of Key

108

To determine if a specified key is present in a dictionary use the **in** keyword:

```
if "8" in dict:
    print("Yes, it is one of the keys in the dictionary")
else:
    print ('not in the list')
```

```
>>>
==== RESTART: C:\Users\Hera Noor\AppData\Local
Yes, it is one of the keys in the dictionary
>>> |
```

Change and Update

109

You can change the value of a specific item by referring to its key name.

```
>>> dict
{'1': 'corolla', '2': 'alto', '3': 'ferrari', '4': 'altis', '5': 'cultus', '6':
'city', '7': '1993', '8': 'Alto VXL', 'year': 1947}
>>> dict['4']='apple'
>>> dict
{'1': 'corolla', '2': 'alto', '3': 'ferrari', '4': 'apple', '5': 'cultus', '6':
'city', '7': '1993', '8': 'Alto VXL', 'year': 1947}
```

The **update()** method will update the dictionary with the items from the given argument. The argument must be a dictionary, or an iterable object with key:value pairs.

```
>>> dict.update({'4': 'altis'})
>>> dict
{'1': 'corolla', '2': 'alto', '3': 'ferrari', '4': 'altis', '5': 'cultus', '6':
'city', '7': '1993', '8': 'Alto VXL', 'year': 1947}
>>> |
```

Remove Items

110

There are several methods to remove items from a dictionary.

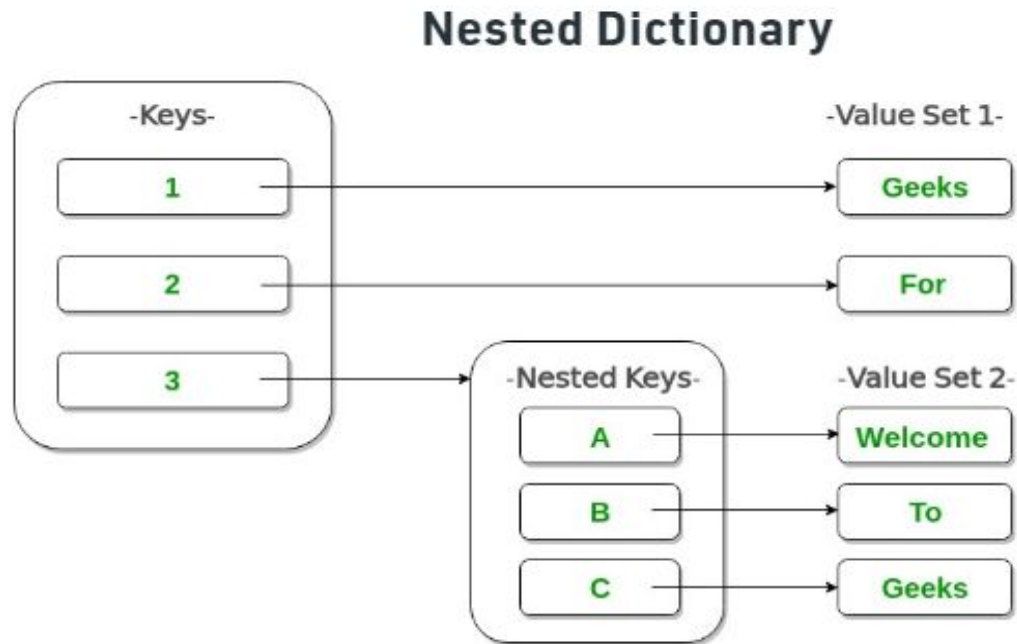
- The **pop()** method removes the item with the specified key name
- The **popitem()** method removes the last inserted item (in versions before 3.7, a random item is removed instead).
- The **del** keyword removes the item with the specified key name. The del keyword can also delete the dictionary completely.
- The **clear()** method empties the dictionary.

```
>>> dict
{'1': 'corolla', '2': 'alto', '3': 'ferrari', '4': 'altis', '5': 'cultus', '6':
'city', 'color': 'green'}
>>> dict.pop('color')
'green'
>>> dict.popitem()
('6', 'city')
>>> del dict['1']
>>> dict
{'2': 'alto', '3': 'ferrari', '4': 'altis', '5': 'cultus'}
>>> dict.clear()
>>> dict
{}
```

Nested Dictionary

111

The dictionaries can be nested, like dictionary within dictionary. For this nested keys are defined.



Nested Dictionary

112

A dictionary can contain dictionaries, this is called nested dictionaries.

```
>>> myfamily = {  
    "child1" : {  
        "name" : "Zuha",  
        "year" : 2009  
    },  
    "child2" : {  
        "name" : "Mohammad Ahmed",  
        "year" : 2014  
    },  
    "child3" : {  
        "name" : "Hasan",  
        "year" : 2017  
    }  
}
```

- **clear()** – Remove all the elements from the dictionary
- **copy()** – Returns a copy of the dictionary
- **get()** – Returns the value of specified key
- **items()** – Returns a list containing a tuple for each key value pair
- **keys()** – Returns a list containing dictionary's keys
- **fromkeys()** – Returns a dictionary with the specified keys and value
- **pop()** – Remove the element with specified key
- **popitem()** – Removes the last inserted key-value pair
- **update()** – Updates dictionary with specified key-value pairs
- **values()** – Returns a list of all the values of dictionary
- **setdefault()** – Returns the value of the specified key. If the key does not exist: insert the key, with the specified value.
- The **del** keyword removes the item with the specified key name.

- Task 1: Generate a code which take a list of numbers and provide square of these numbers in a new list.
Numbers = [1,2,3,4]
- Task 2: Write a code which take list of numbers as in input from user and check generate a list of numbers which is divisible by 2. Those number which are divisible by 2 will be saved in new list.
- Task 3: Write a code which take two list of numbers (l1,l2) as in input and find the common elements and save it in new list.
l1 = [1,2,3,4] l2 = [1,2,8,9]

- Task 1 : Write a code which provides a dictionary of cubic values. Key is number, value is the cube as follows:
 $\{1:1, 2:8, 3:27,\}$
- Task 2: Write a code which takes four inputs from user name, age, fav_sports as a list and fav_hobbies also as a list and print them in the following format.
name= ameem
age= 30
fav_sport= ['cricket', ' football']
fav_hobbies= ['eating', ' playing']

Functions

A function is a groups of statements made to execute them more than once in a program. A function has a name.

Functions can compute a result value and can have parameters that serve as function inputs which may differ each time when function is executed.

Functions are used to:

- Reduce the size of code as it increases the code reusability
- Split a complex problem in to multiple modules (functions) to improve manageability

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Components of a function:

- Function code - is something that is written inside function or function definition.
- Function call is mandatory for the execution of function.

Function Components:

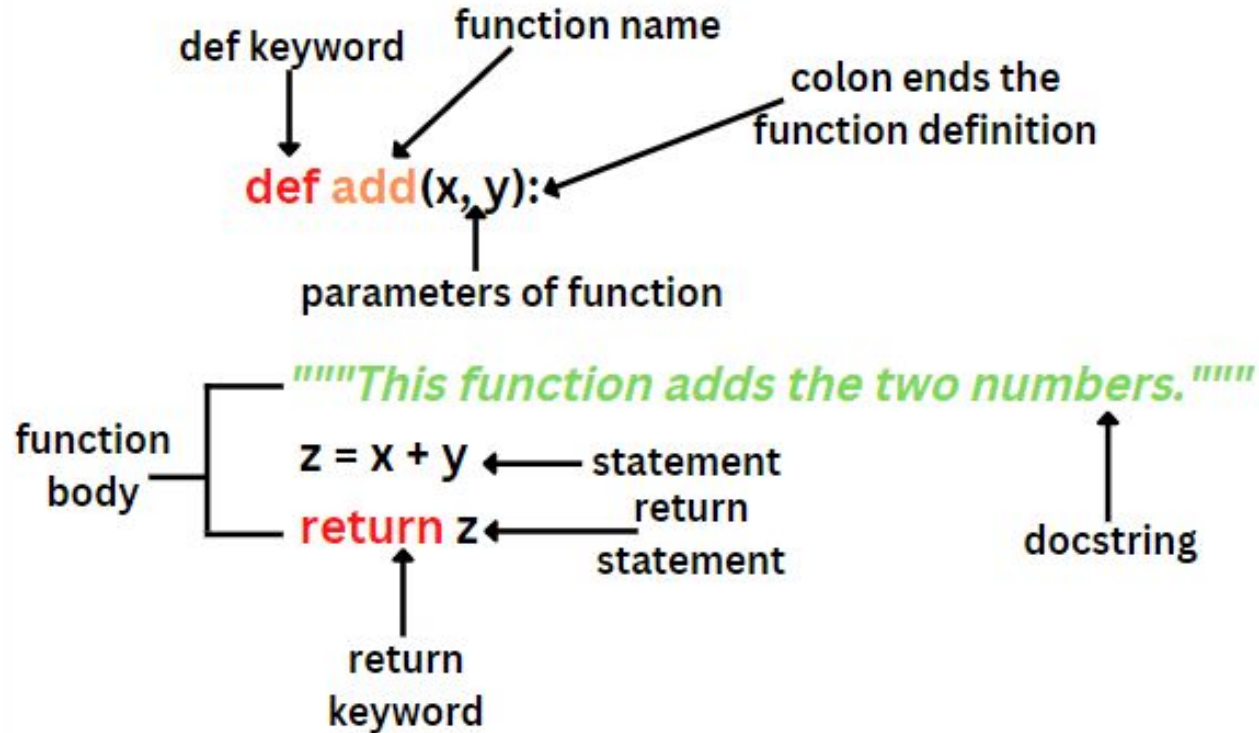
- Function Signature
 - * Function Name
 - *Function Arguments (optional)
- Doc string
- Function Body
- Function Return Statement

Syntax:

```
def function_name (arguments):  
    doc string  
    body  
    return statement
```

Function can be used without argument. E.g. Function of length, it accept argument and then return length of the argument.

Function body is the code.
Return statement if needed.



Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma. The terms parameter and argument can be used for the same thing: i.e. information that are passed into a function.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
def my_function(fname):  
    print(fname + " Apple")  
  
my_function("Red")  
my_function("Yellow")  
my_function("Green")
```

Number of Arguments

122

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

```
def my_function(fname, lname):  
    print(fname + ' ', lname)  
  
my_function("Red", 'apple')  
my_function("Yellow", 'banana')  
my_function("Green", 'chilli')
```

Default Parameter Value

123

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value:

```
def my_function(country = "Pakistan"):  
    print("I am from " + country)
```

```
my_function("Karachi")  
my_function("Lahore")  
my_function()  
my_function("DI khan")
```

```
= RESTART: C:\Users\He:  
8\def.py  
I am from Karachi  
I am from Lahore  
I am from Pakistan  
I am from DI khan  
^^^ |
```

Functions - Example

124

Write a function that takes radius of a circle from user and returns its circumference.

```
def circle (r):  
    return 2*3.14*r  
  
>>> print(circle(3))  
9.42  
>>> x=7  
>>> print(circle(x))  
21.98  
>>> r=3  
>>> print(circle(r))  
9.42
```

Understanding doc function

```
def circle (r):  
    '''returns the circumference of circle'''  
    return 2*3.14*r  
print(circle.__doc__)
```

Docstrings:

- Python docstrings are the string literals that appear right after the definition of a function, method, class, or module.

Functions - Example

125

Modifying previous example to return area of circle via variables.

```
def circle (r):  
    area=3.14*r**2  
    circumference=r*3.14  
    return area, circumference
```

```
>>> print(circle(5))  
(78.5, 15.700000000000001)  
>>> x=7  
>>> print(circle(x))  
(153.86, 21.98)  
>>> r=3  
>>> print(circle(r))  
(28.26, 9.42)  
>>> |
```

Accessing values of tuples.

```
>>> result=circle(5)  
>>> print('area=', result[0], 'and',  
'circumference=', result[1])  
area= 78.5 and circumference= 15.700000000000001
```

To print the area and Circumference

```
def circle (r):  
    print('circumference=', r*3.14)  
    print('area=', 3.14*r**2)|  
>>> r=5  
>>> circle(r)  
circumference= 15.700000000000001  
area= 78.5  
>>> circle(7)  
circumference= 21.98  
area= 153.86
```

Help function

126

The help function is used to view documentation

```
>>> help(circle)
Help on function circle in module __main__:

circle(r)

>>> help(print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file: a file-like object (stream); defaults to the current sys.stdout.
    sep:  string inserted between values, default a space.
    end:  string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.

>>> help(int)
```

Squeezed text (266 lines).

Positional Arguments

- Must be put in correct order when calling the function.
- Do not have any default value called args.
- If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

```
def my_function(*kids):  
    print("The youngest child is " + kids[0])  
  
my_function("Hadi", "Rahim", "Tina")
```

Keyword Arguments

- Include a keyword and equal sign.
- Used to set defaults; have some default value.
- Placed after the positional arguments if any.
- Can be placed in any order among other keyword arguments when calling the function. called kwargs.

Keyword Arguments and Arbitrary Keyword Arguments¹²⁸

Keyword Arguments: You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter.

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child1)  
  
my_function(child1 = "Hadi", child2 = "Wara", child3 = "Rahim")
```

Arbitrary keyword Argument: If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.

```
def my_function(**kid):  
    print("His last name is " + kid["lname"])  
  
my_function(fname = "Hadi", lname = "Rehman")
```

VLA *args & VLA *Kwarg

129

Example: The following code used a function to print all objects passed to it, on separate lines.

```
def my_print(*args):  
    for value in args:  
        print(value)  
my_print('Learning', 'Python', '3')  
print()  
my_print('Learning', 'Python', '3', 'and', 'Java', 'too')
```

```
Learning  
Python  
3  
  
Learning  
Python  
3  
and  
Java  
too
```

```
def my_func(**kwargs):  
    for item in kwargs.items():  
        print(item)  
my_func(first='Computer', last='Programming')
```

```
('first', 'Computer')  
('last', 'Programming')
```

- A Python module is a py file containing functions and classes.
 - useful code that eliminates the need for writing codes from scratch.
- A package or library is a collection of modules.
- Python has a rich set of libraries.
 - There are over 137,000 python libraries present today.
- Python libraries play a vital role in developing machine learning, data science, data visualization, image and data manipulation applications and more.

Module, Packages and Libraries

131

BuiltIn Modules	Standard Library	Third Party Library
It contains all the core built-in types/classes and functions. Loaded automatically when the interpreter starts. It does not contain all the functions and classes supported by Python. - size is kept small for efficiency. Some functions of builtins module are: - min() - max() - len() - print() Other modules are kept in the library called The Standard Library.	It is the native library of Python that comes with Python installation. The modules of this library need to be imported if used. Some modules in The Standard Library include: - mathematical functions - date-time functions - graphical user interface (GUI) See https://docs.python.org/ for details on builtins module and the Standard Library	Python has a huge collection of third-party modules/packages available for usage. These modules/packages must be installed and then imported if used. Some commonly used Python third party packages: - Pillow - Numpy – Pandas <u>Package installers:</u> - pip installs Python packages - conda installs packages written in any language

- When the Python interpreter executes an import statement:
 - the corresponding file containing the module is searched.
 - the module code is run to create objects defined in that module.
 - a namespace is created where the objects live.
- Hence a separated namespace is created for every module.
- An application may consist of several modules:
 - top level module is the main program from which the execution starts. It is called main.
 - the remaining modules are library modules that are imported by the top level module.

How to import...

133

Using import statement	Using from with import	Using from to import all functions of a module
Creates a separate namespace Example: <pre>>>> import math >>> math.sqrt(49)</pre> Single statement can be used to import more than one module. <pre>>>> import math, random</pre>	It copies selected functions into the namespace of the importing module. Remaining functions are not instantiated. Functions can be called without the dot operator. Example: <pre>>>> from math import sqrt >>> sqrt(49)</pre>	Example: <pre>>>> from math import * >>> sqrt(49)</pre> Now all the functions are copied to the calling namespace and can be used without the dot operator. Not a good idea!! - Takes too much space - Variable names may clash

- Define a function which takes two numbers as an input and return the greater of two numbers.
- Define is_palindrome function that takes one word in string as an input and return True, if it is palindrome else return false. (Words like Madam, naman)
Hint: reverse the string and compare it with itself
- Define a function which takes two arguments like (power , *args) and list of numbers args as an input and return a list with power of numbers which is given as an input
Input. (2, [3,4,5])
Output. [9, 16, 25]

Namespaces

- Namespace is an abstract container or environment created to hold a logical grouping of identifiers / variables.
- An identifier/variable defined in a namespace is usually associated only with that namespace.
- The same identifier/variable can be independently defined in multiple namespaces.
- An identifier/variable defined in one namespace may or may not interfere with the identifier/variable defined in another namespace.
- Languages that support namespaces specify the rules that determine to which namespace an identifier/variable belongs.

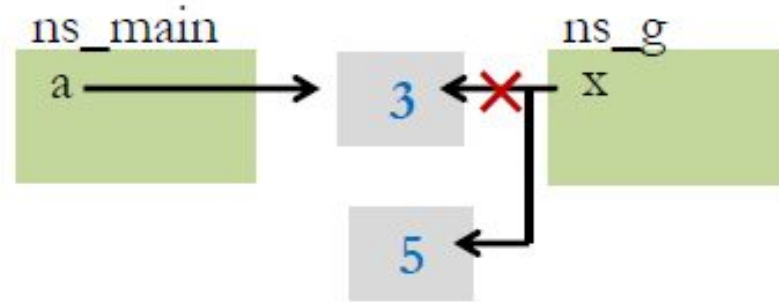
Parameter passing in Namespaces

137

Passing Immutable Objects as Parameters

Example 1: Consider the following code:

```
def g(x): Step 3  
    x=5 Step 4  
a=3 Step 1  
g(a) Step 2  
print(a) Step 5
```



Output
3

Problem

138

Write a function that inputs a list of integers from user.

```
def lst_input():  
    my_str=input('Enter integer values separated by commas:')  
    my_str=my_str.split(',')  
    int_lst=[]  
    for i in my_str:  
        int_lst.append(int(i))  
    return int_lst  
  
print('You entered:',lst_input())
```

Execution starts here

Diagram illustrating the execution flow and variable states:

- `my_str` is assigned the string `'1,2,3,4'`.
- `my_str` is split into the list `['1', '2', '3', '4']`.
- The loop iterates over the list, and each element is converted to an integer and appended to `int_lst`.
- The final state of `int_lst` is `[1, 2, 3, 4]`.

Output

Enter integer values separated by commas: 1,2,3,4

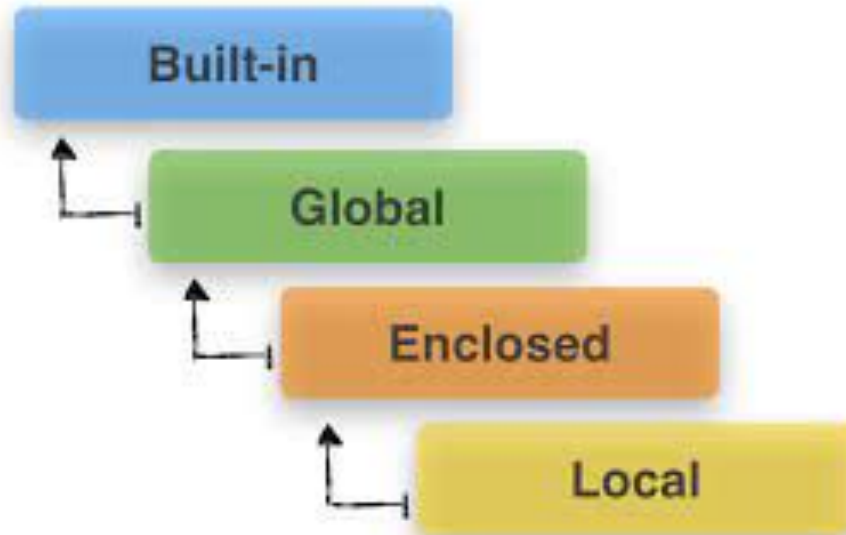
You entered: [1,2,3,4]

The Python interpreter searches a name (variable or function) in the following order:

1. Search the local namespace – L
2. Search the enclosing function's namespace (for nested functions) – E
3. Search the global namespace – G
4. Search the namespace of module builtins – B

The builtin module is automatically loaded every time Python interpreter starts.

- It is the top level execution environment.
- It provides direct access to all built-in identifiers (types/classes and functions) of Python.



The LEGB Rule

141

Example:

```
def f(b):  
    a=b  
    return a*b
```

```
x=3  
print(f(x))  
print(x)
```

- `f` has **global** namespace as it is defined in the main module.
- `a` and `b`, defined in `f()` have **local** scopes.
- `x` defined in main module has **global** namespace.
- `print()` function is defined in **builtins** modules.

Output

9
3

ns_main (global)

f
x

ns_f (local)

a
b

ns_print (module builtins)

print

- Names assigned inside a def can only be seen by the code within that def. You cannot even refer to such names from outside the function.
- Names assigned inside a def do not clash with variables outside the def, even if the same names are used elsewhere. A name X assigned outside a given def (i.e., in a different def or at the top level of a module file) is a completely different variable from a name X assigned inside that def.
- If a variable is assigned inside a def, it is local to that function.
- If a variable is assigned in an enclosing def, it is nonlocal to nested functions.
- If a variable is assigned outside all defs, it is global to the entire file.

Scope – Local vs Global

143

Local Scope: names assigned inside a function.

- such variables are called local variables.

Global Scope: names assigned in the interpreter shell or main module (outside of any function).

- such variables are called global variables.

Advantages of local variables

Functional independence – same variable name can be used in different functions.

Does not occupy memory unnecessarily.

Advantages of global variables

Can be used to define constants in a program.

The Global keyword

144

The keyword called global used inside a function allows the function to update a global variable.

Practice Problem: Modifying the last code on the previous slide.

```
def h():  
    global x  
    x+=1 ← Now this is the same global x accessed from this local namespace  
    print(x)
```

```
x=5  
h()  
print(x)
```

Output

6

6

Scope example

145

Example

Global scope

X = 99 # X and func assigned in module: global

def func(Y): # Y and Z assigned in function: locals

Local scope

 Z = X + Y # X is a global

return Z

func(1) # func in module: result=100

Recursion

A recursive function is a function that calls itself.

The idea is to represent a problem in terms of one or more smaller problems.

Components / properties of a recursive function:

Base Case

- Indicates the stopping condition.
- Could be more than one.

Recursive Call

- Moves the execution towards the base case.
- Could be more than one.

Recursion with a Base Case

148

Example: Write a factorial function using recursion

```
def factorial(n):  
    if (n==1 or n==0):  
        return 1  
    else:  
        return (n * factorial(n - 1))  
  
num = 5;  
print("number : ", num)  
print("Factorial : ", factorial(num))
```

```
=====  
number : 5  
Factorial : 120  
>>>|
```

Base Case: The function returns 1 when n is 0 or 1.

Recursive Call: The function multiplies n by the factorial of $n - 1$

Why Use Recursion??

149

- Recursion is made for solving problems that can be broken down into smaller, repetitive problems.
- It is especially good for working on things that have many possible branches and are too complex for an iterative approach.
- Recursion may be efficient from programmer's point of view.
- Recursion may not be efficient from computer's point of view.

Files

A file is a sequence of bytes stored on a secondary memory device.

Files are of various types:

- Text Files
- Spreadsheets
- Binary Files
- Executable Files

Files are managed by File system that operating system supports.

Processing a file is based on three steps:

- Operating a file for reading or writing
- Reading from or writing to the file
- Closing the file

The function `open()` is used to open the files (text or binary). This function is defined in built-in modules.

The function takes:

- A file name (with or without path)
- ‘\’ is used for path but since it may coincide with escape sequence so python accepts ‘/’ (forward slash).
- A path could be absolute or relative:
 - Raw / absolute path starts from root directory:
e.g.: `c:\office\classes\CP\Text.txt`
 - Relative path starts the sequence from current directory:
e.g: `CP\Text.txt`

The function takes:

- A file name (with or without path).
- Mode specifies how to interact with opened file.
- r=>reading mode (default)
- w=>writing mode, if the file already exists otherwise its content is wiped out
- a=>append mode, the data will append to the end of file.
- t=>text mode (default)
- b=>binary mode

Opening a file

154

```
>>> f=open('myfile.txt')  $\equiv$  f=open('myfile.txt', 'r')
```

- Opens myfile.txt if it exist
- Generates error if the file does not exist.
- The file is opened for reading only.

```
>>> f=open('myfile.txt', 'w')
```

- Opens myfile.txt for writing.
- Creates a new file if it doesn't exist.
- Overwrites the existing file.

Opening a file

155

```
>>> f=open('myfile.txt', 'a')
```

- Opens myfile.txt for writing.
- Creates a new file if it doesn't exist.
- Appends at the end of existing file.

- Automatically closes a file after block of code is executed.

Syntax:

```
with open <file name> as f:  
    <block>
```

- Opens <file name> and assigns it handler.
- Closes f after <block> is executed.

f.read(n)

- reads and returns as string 'n' characters from file
- 'f' or until the end of file is reached.

f.read()

- reads and returns as string characters from file f until the end of file.

f.readline()

- reads and returns as string characters from file f until (including) new line character or end of file.

f.readlines()

- reads and returns as list.

- With every opened file, the system will associate a cursor that points to the character in the file.
- When the file is first opened, the cursor typically points to the start of the file.
- Using different types of read operations consecutively, second read commences from where first read ended.

- `f.name`

Contains name of file. It's an attribute, not a method.

- `f.seek(offset, from_what)`

Changes file object position.

Position is computed from adding offset to a reference point

Reference point is selected by `from_what` argument.

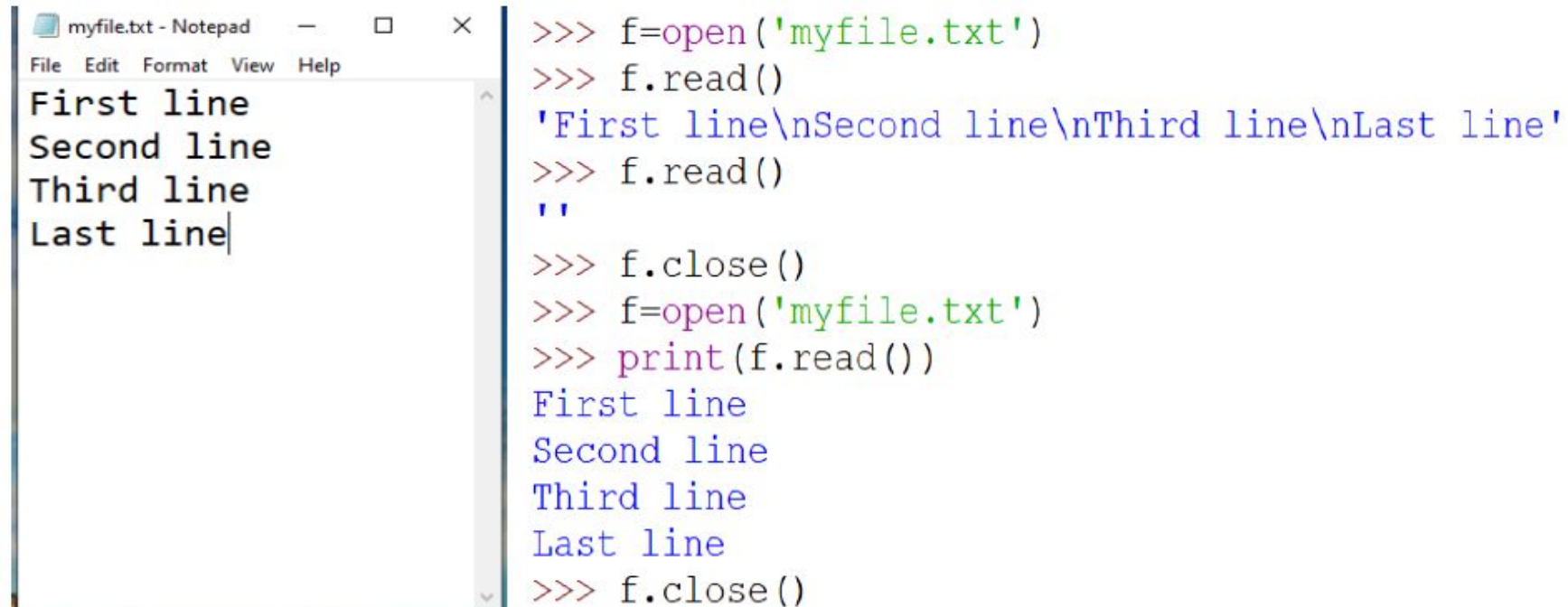
- `From_what=0`, offset measured from start of file.
- `From_what=1`, offset measured from current position.
- `From_what=2`, offset measured from EoF
- Default value is 0

- `f.tell()`: returns an integer giving file objects current position in the file as number of bytes from the beginning of the file.

Example -1

160

Store the following file as myfile.txt



The image shows a side-by-side comparison of a text file's content and the Python code used to read it. On the left, a Notepad window titled 'myfile.txt - Notepad' displays the text: 'First line', 'Second line', 'Third line', and 'Last line'. On the right, a Python terminal shows the following code and its output:

```
>>> f=open('myfile.txt')
>>> f.read()
'First line\nSecond line\nThird line\nLast line'
>>> f.read()
''
>>> f.close()
>>> f=open('myfile.txt')
>>> print(f.read())
First line
Second line
Third line
Last line
>>> f.close()
```

Example -2

161

Reading one line at a time

```
File Edit Format View Help
First line
Second line
Third line
Last line|
```

```
>>> f=open('myfile.txt')
>>> f.readline()
'First line\n'
>>> f.readline()
'Second line\n'
>>> f.readline()
'Third line\n'
>>> f.close()
```

Example - 3

162

Reading all lines

```
>>> f=open('myfile.txt')
>>> f.readlines()
['First line\n', 'Second line\n', 'Third line\n', 'Last line']
>>> f=open('myfile.txt')
>>> p=f.readlines()
>>> print(p[1])
Second line

>>> f.close()
```

Example - 4

163

```
>>> f=open('myfile.txt')
>>> f.read()
'First line\nSecond line\nThird line\nLast line'
>>> f.read()
''
>>> f.seek(0)
0
>>> f.read()
'First line\nSecond line\nThird line\nLast line'
>>> f.seek(3)
3
>>> f.read(5)
'st li'
>>> f.tell()
8
```

- Python does not have a `random()` function to make a random number, but Python has a built-in module called `random` that can be used to make random numbers.
- You can import `random` module up to any number.

```
>>> import random
>>> print(random.randrange(3,7))
4
>>> print(random.randrange(2,9))
5
>>> print(random.randrange(13,100))
47
>>> print(random.randrange(1,100))
34
>>> print(random.randrange(1,100))
80
.
```

Debugging

Types of Error

166

- There are generally two types of errors namely syntax and logical errors.

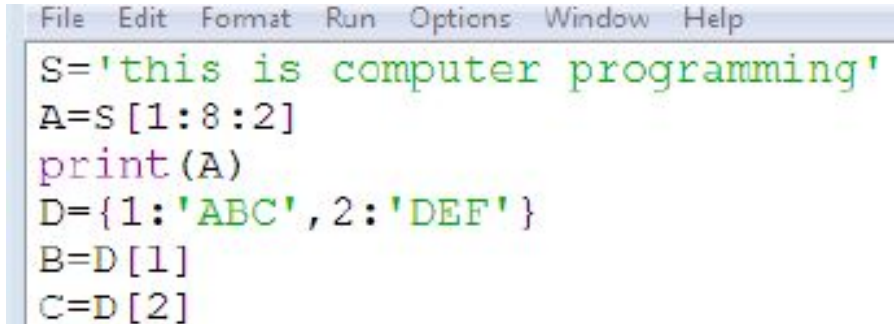
LOGICAL ERROR	SYNTAX ERROR
Logical errors occur when a program does not do what the programmer expects it to do.	Syntax errors are usually easy to fix because the compiler can detect these errors.

- The logical errors might only be noticed during runtime. Because logical errors are often hidden in the source code, they are typically harder to find than syntax errors.

- The process of finding out defects (logical errors) in the program and fixing them is known as debugging.
- Debugging is an integral part of the programming process.

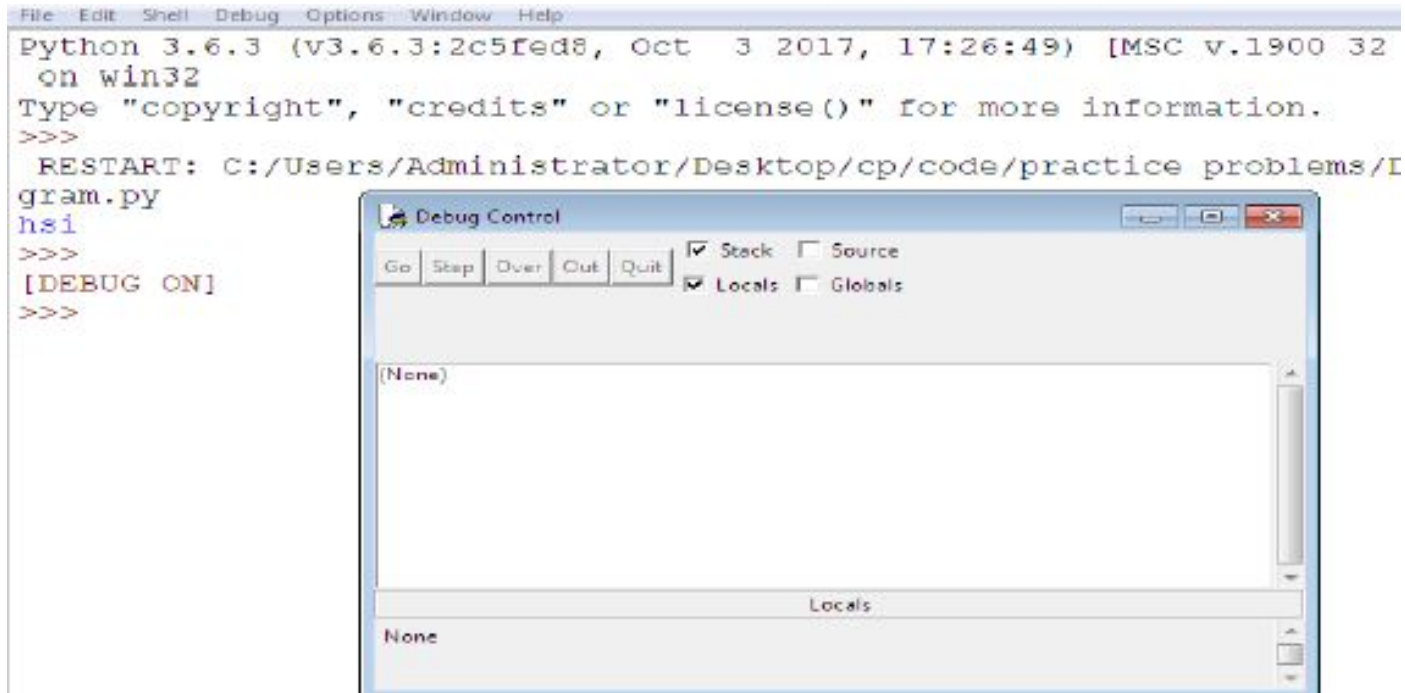
Program Debugging With IDLE

1. Open Python shell
2. Go to file>New and open a python script file
3. Write a program on that file

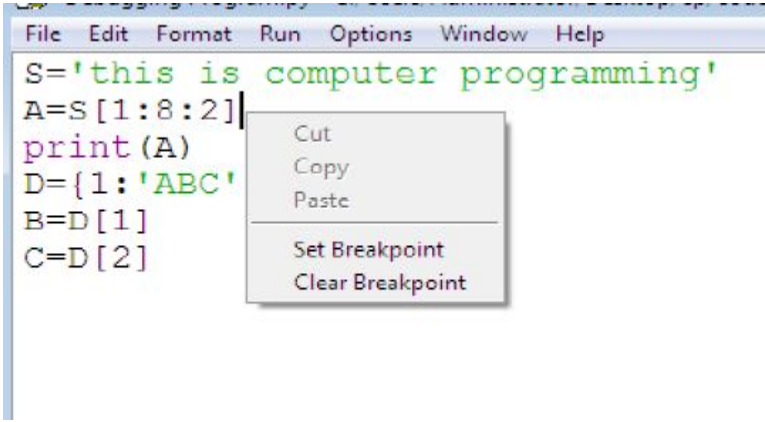


```
File Edit Format Run Options Window Help
S='this is computer programming'
A=S[1:8:2]
print(A)
D={1:'ABC',2:'DEF'}
B=D[1]
C=D[2]
```

4. Go to Python Shell and select the Debug option. A window will appear as shown below

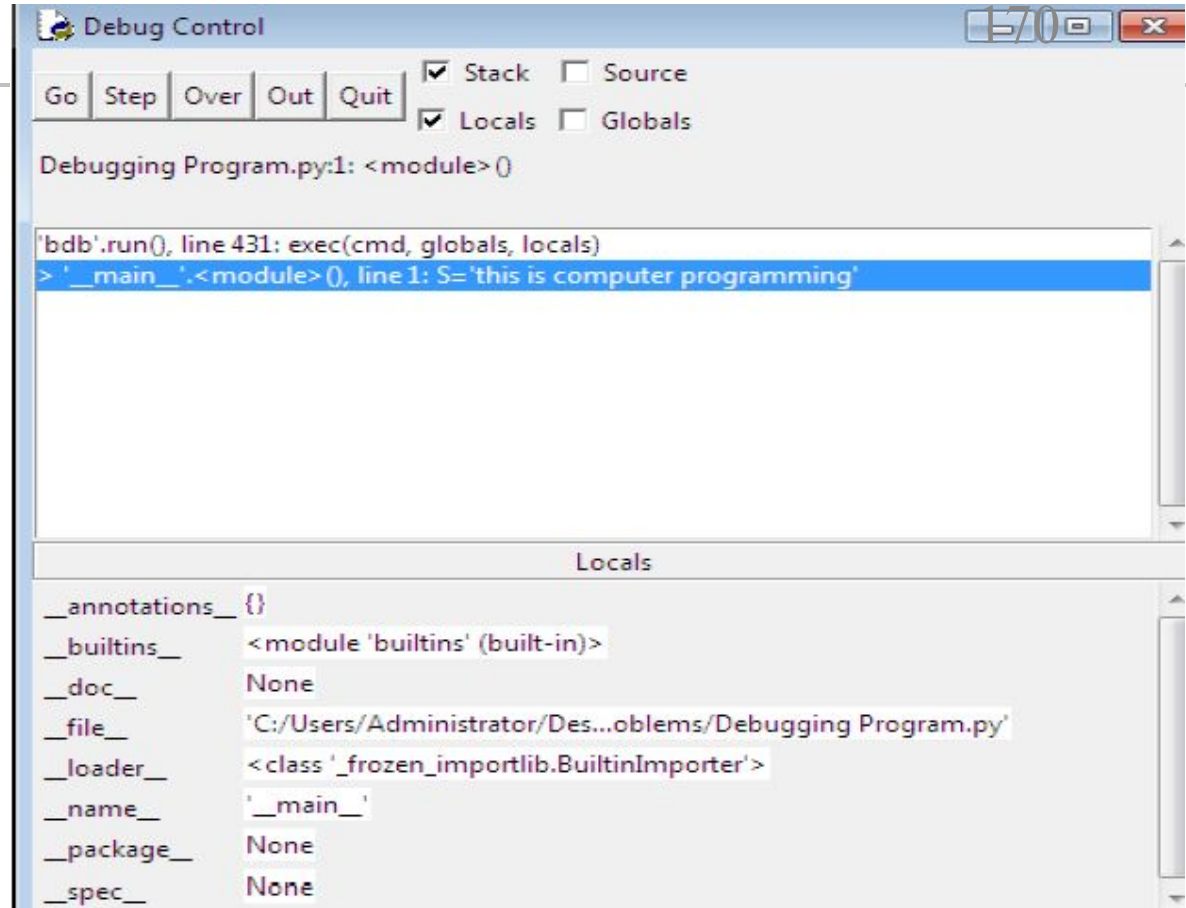


5. Set break point by right clicking on the particular line.



Debugging

6. Now go to python script and click Run and notice the line highlighted by blue. Note that the Debug Control window is opened and that the blue line states that the line 1 "S='this is computer programming'" is ready to be executed.



NOTE

Go	It will make the program run at normal speed until a breakpoint is encountered (or input is requested or the program finishes).
Step	It is used to step through your code, one line at a time.
Over	If the statement to be executed has a function call in it, go off and do the function call without showing any details of the execution or variables, then return and give the human control again, "step over the function"
Out	Assumes you are in some function's code, finish execution of the function at normal speed, return from the function and then give the human control again, "step out of the function"
Quit	the execution of the entire program

Summary

- Setting breakpoints
- Stepping through the source code one line at a time
- Inspecting the values of variables as they change
- Making corrections to the source as bugs are found
- Rerunning the program to make sure, the fixes are correct